

Training Course Design & Student Evaluation Multi-Agent System

Designed by: Mohammad Abu Alqassem

Supervised by: Dr. Fakh El-Din Saeed

المرحلة الثانية من المشروع : تصميم الـ Pydantic Schemas.

ننتقل الآن إلى الخطوة الثانية: تصميم الـ Pydantic Schemas.

هذه الخطوة مهمة جداً؛ لأن الـ Schemas ستكون العقد (Contracts) بين الـ Agents. أي أن كل Agent يعرف بالضبط ماذا يستقبل وماذا يجب أن ينتج، ولن نعتمد على نص غير منظم.

المرحلة Pydantic Schemas2

سنضع الـ Schemas داخل:

```
training_course_agent/  
└─ schemas/  
   └─ __init__.py  
   └─ course.py  
   └─ assessment.py  
   └─ student.py  
   └─ analytics.py  
└─ recommendation.py
```

وسنبدأ بالـ Schemas الأساسية.

1. Course Request :

هذا يمثل طلب المدرب. مثلاً:

Create a 30-hour Python course for beginners.

نحوّله إلى:

```
from pydantic import BaseModel, Field  
  
class CourseRequest(BaseModel):  
    title: str = Field(min_length=1)  
    level: str = Field(min_length=1)  
    duration_hours: float = Field(gt=0)
```

```
student_count: int = Field(gt=0)
goals: list[str] = Field(default_factory=list)
prerequisites: list[str] = Field(default_factory=list)
```

مثال:

```
request = CourseRequest(
    title="Python Programming",
    level="Beginner",
    duration_hours=30,
    student_count=20,
    goals=[
        "Understand Python fundamentals",
        "Write basic Python programs",
        "Use functions and data structures"
    ],
    prerequisites=[
        "Basic computer skills"
    ]
)
```

2. Learning Objective:

كل Module يجب أن يحتوي على Learning Objectives.

```
class LearningObjective(BaseModel):
    id: str
    description: str
    level: str
```

مثلاً:

LO-001

Understand Python variables

Level: Understand

نستخدم id حتى نستطيع لاحقاً ربط:

Question → Learning Objective

Student Result → Learning Objective

Recommendation → Learning Objective

وهذا مهم جداً في التحليل.

3. Module :

```
class Module(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    description: str
```

```
    duration_hours: float = Field(gt=0)
```

```
    objectives: list[LearningObjective]
```

مثلاً:

Module:

M-001

Title:

Python Variables

Duration:

4 hours

Objectives:

LO-001

LO-002

LO-003

4. CoursePlan :

هذا أهم Schema في المرحلة الأولى.

```
class CoursePlan(BaseModel):  
    title: str  
    description: str  
    level: str  
    duration_hours: float = Field(gt=0)  
    prerequisites: list[str] = Field(default_factory=list)  
    modules: list[Module]
```

وبالتالي:

Course Designer Agent

|
|



CoursePlan

يصبح الناتج قابلاً للتحقق برمجياً.

5. GateResult :

نحتاج Schema موحدًا لكل الـ Gates.

```
class GateResult(BaseModel):  
    passed: bool  
    gate_name: str  
    errors: list[str] = Field(default_factory=list)
```

مثلاً إذا كان Course صالحاً:

```
passed = True  
gate_name = "course_validation"  
errors = []
```

```
passed = False
```

```
gate_name = "course_validation"
```

```
errors:
```

```
- Module durations do not equal course duration
```

6. AgentTask :

نحتاج أيضًا Schema للـ Orchestrator.

```
class AgentTask(BaseModel):
```

```
    next_agent: str
```

```
    task: str
```

```
    reason: str
```

```
    required_input: list[str]
```

مثلاً:

```
{
    "next_agent": "course_designer",
    "task": "Create a beginner Python course",
    "reason": "The user requested a new course",
    "required_input": [
        "course_request"
    ]
}
```

وهذا أفضل بكثير من جعل Orchestrator يعيد نصًا مثل:

```
I think the course designer should handle this.
```

7. Assessment Schema

بعد ذلك نحتاج إلى الاختبارات.

```
class Question(BaseModel):
```

```

id: str
question: str
question_type: str
difficulty: str
objective_id: str
options: list[str] = Field(default_factory=list)
correct_answer: str
marks: float = Field(gt=0)

```

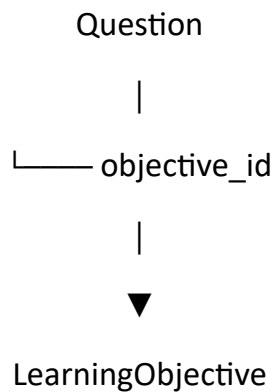
ثم:

```

class Assessment(BaseModel):
    id: str
    title: str
    questions: list[Question]
    total_marks: float = Field(gt=0)

```

لاحظ العلاقة المهمة:



وهذه ستسمح لنا بقياس:

هل الطالب أتقن Learning Objective معيناً؟

8. Student Schema :

```

class Student(BaseModel):
    id: str
    name: str

```

course_id: str

ثم إجابة الطالب:

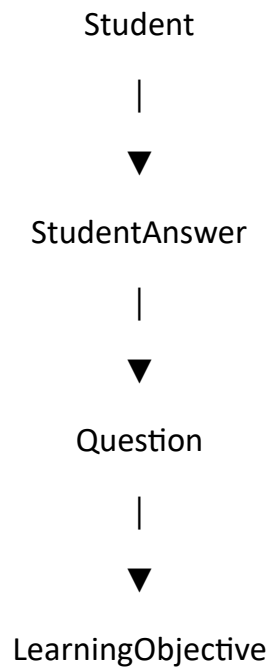
```
class StudentAnswer(BaseModel):
```

student_id: str

question_id: str

answer: str

وبذلك:



9. Student Evaluation

بعد التصحيح نحتاج إلى نتيجة لكل Learning Objective.

```
Class ObjectiveResult(BaseModel):
```

objective_id: str

score: float = Field(ge=0, le=100)

mastery_level: str

ثم:

```
class StudentEvaluation(BaseModel):
```

student_id: str

assessment_id: str

overall_score: float = Field(ge=0, le=100)

objective_results: list[ObjectiveResult]

مثلاً:

Student: S001

Overall:

68%

LO-001:

90% → Mastered

LO-002:

80% → Mastered

LO-003:

55% → Needs Improvement

LO-004:

40% → Weak

10. Learning Analytics :

الآن نحتاج إلى تحليل أداء المجموعة.

class LearningGap(BaseModel):

objective_id: str

mastery_score: float = Field(ge=0, le=100)

severity: str

ثم:

class LearningAnalysis(BaseModel):


```
class_average: float = Field(ge=0, le=100)
strong_objectives: list[str] = Field(default_factory=list)
weak_objectives: list[str] = Field(default_factory=list)
learning_gaps: list[LearningGap] = Field(default_factory=list)
```

مثلاً:

Class Average: 71%

Strong:

LO-001

LO-002

Weak:

LO-003

LO-004

Learning Gaps:

LO-004 → 42% → Critical

11. Recommendation Schema :

آخر مرحلة هي التوصيات.

```
class Recommendation(BaseModel):
```

```
    id: str
```

```
    target: str
```

```
    issue: str
```

```
    action: str
```

```
    priority: str
```

```
    rationale: str
```

ثم:

```
class RecommendationReport(BaseModel):  
    recommendations: list[Recommendation]
```

مثلاً:

Recommendation:

R-001

Target:

LO-004

Issue:

Low mastery

Action:

Reteach Python Functions

Priority:

High

Rationale:

Class mastery is below the required threshold.

12- العلاقة الكاملة بين Schemas

الآن أصبح لدينا:

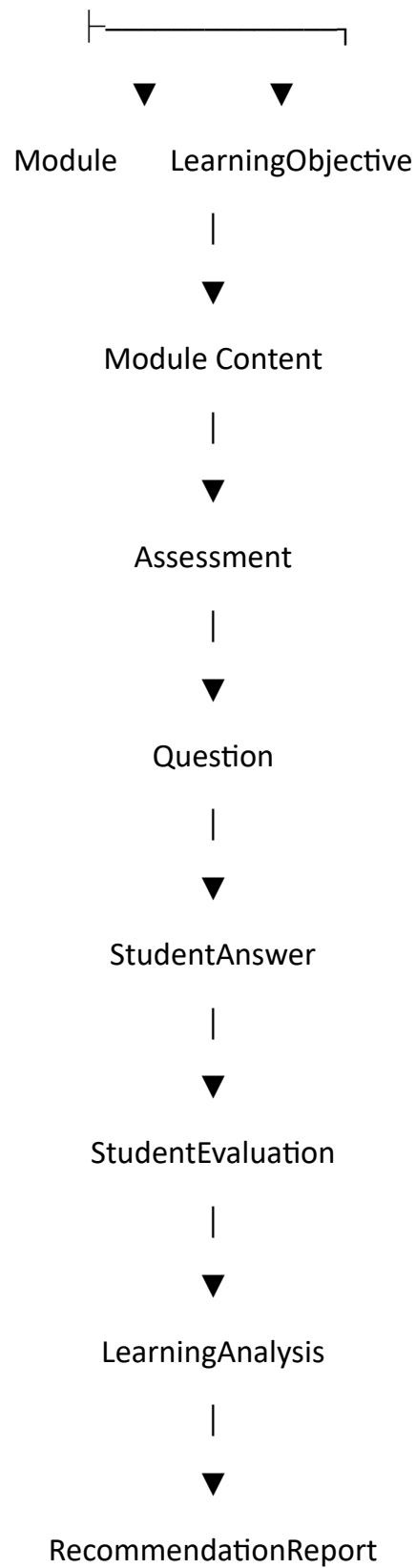
CourseRequest

|



CoursePlan

|



13. نقطة مهمة جداً Validation Gates :

سنضيف الآن قواعد تحقق تتجاوز مجرد Pydantic.

مثلاً Pydantic يستطيع التأكد أن:

`duration_hours > 0`

لكن لا يستطيع وحده أن يضمن أن:

Course Duration = Sum(Module Durations)

Domain Gates. لذلك سنحتاج

مثلاً:

```
def validate_course_duration(course: CoursePlan) -> GateResult:
```

```
    total = sum(
        module.duration_hours
        for module in course.modules
    )

    if total != course.duration_hours:
        return GateResult(
            passed=False,
            gate_name="course_duration",
            errors=[
                f"Expected {course.duration_hours} hours "
                f"but modules contain {total} hours."
            ]
        )

    return GateResult(
        passed=True,
        gate_name="course_duration"
    )
```

وهنا يظهر الفرق بين:

Agent Guardrails / Gates و **Schema Validation**

وهذا سيكون جزءاً أساسياً في مشروعنا.

```
training_course_agent/  
|  
|— schemas/  
| |— __init__.py  
| |— course.py  
| |— assessment.py  
| |— student.py  
| |— analytics.py  
| └─ recommendation.py  
|  
|— agents/  
|  
|— tools/  
|  
|— gates/  
|  
|— prompts/  
|  
|— tracing/  
|  
|— evaluation/  
|  
|— database/  
|  
└─ main.py
```

15- أول اختبار : قبل إنشاء أي Agent ، يجب أن نتأكد أن الـ Schemas تعمل.

سننشئ:

```
tests/  
└─ test_schemas.py
```

ونختبر:

Test 1 : هل CourseRequest صحيح؟

Test 2 : هل CoursePlan صحيح؟

Test 3 : هل Pydantic يرفض: duration_hours = -10

Test 4 : هل يرفض Score أكبر من 100؟ score = 150

Test 5 : هل Gate يرفض Course إذا كان: Course = 30 hours و Modules = 20 hours

الخطوة التي سننفذها الآن

بدلاً من الانتقال مباشرة إلى جميع الـ Agents، سنبنى أول جزء قابل للتشغيل فعلياً:

```
schemas/  
course.py  
↓  
gates/  
course_gates.py  
↓  
agents/  
course_designer.py  
↓  
main.py
```

وسيكون أول Agent لدينا: **Course Designer Agent** وسنقوم بتصميمه كاملاً:

Goal → Input → Output → System Prompt → ModelClient → Pydantic → Gate → Retry →
Failure Handling → Trace

ثم نجربه على:

Create a 30-hour beginner Python Programming course for 20 students.

وهذه ستكون أول **Vertical Slice** حقيقية في المشروع، وبعد نجاحها نكرر نفس النمط مع بقية الـ Agents.